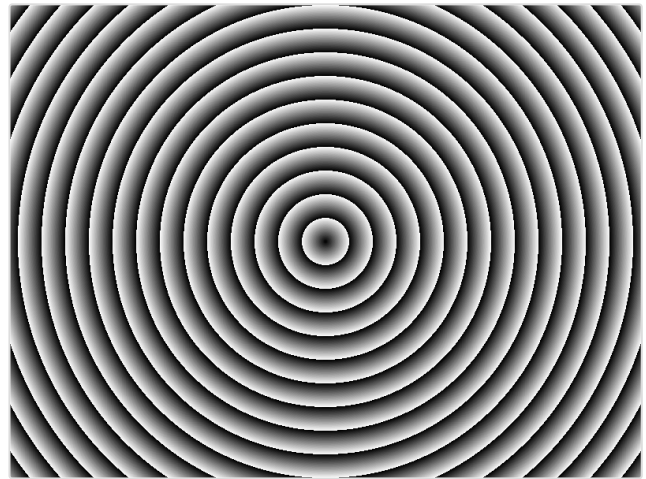
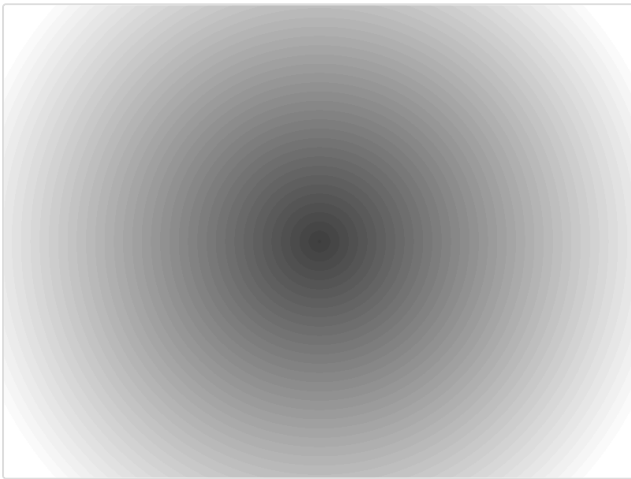
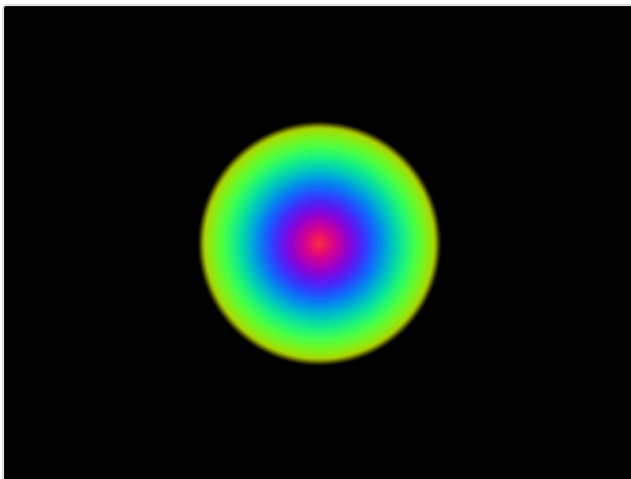


Write-up 1



1. Where the y flip that Lab 05's `viewportTransform()` used to do has gone, which corner of your mode 1 view is green, and the one line you would have to change in a shader copied from ShaderToy.

Lab 05 flipped Y in `viewportTransform` so that “up” in math space landed at the **top** of the picture ($((1 - \text{ndc.y}) / 2) * \text{height}$). That CPU step is gone. Vulkan already numbers pixels from the **top**, so `gl_FragCoord.y = 0` is the top row. Our `p` formula uses that as-is, so `p.y = -1` is the top of the screen.

Mode 1 colours `uv`. Green is `uv.y`, which grows downward, so green is the **bottom-left** corner (pixel `(0, 599)`).

A ShaderToy shader assumes Y grows **up**. In a copy, flip the pixel Y, the same idea as `shadertoy.glsl`: use `u.resolution.y - gl_FragCoord.y` instead of `gl_FragCoord.y`.

2. Why a fixed threshold, `smoothstep(0.01, -0.01, d)`, is resolution dependent while `smoothstep(w, -w, d)` is not. Give the width of each, in pixels, at 800×600 and at 3840×2160 .

`0.01` is a distance in **shape units**, not pixels. One shape unit is `height / 2` pixels: **300** at 800×600, **1080** at 3840×2160. The ramp from `0.01` to `-0.01` is `0.02` shape units, so it is $0.02 \times 300 = 6$ pixels on the lab window and $0.02 \times 1080 = 21.6$ pixels at 4K. Same number in the shader, thicker edge on a bigger screen.

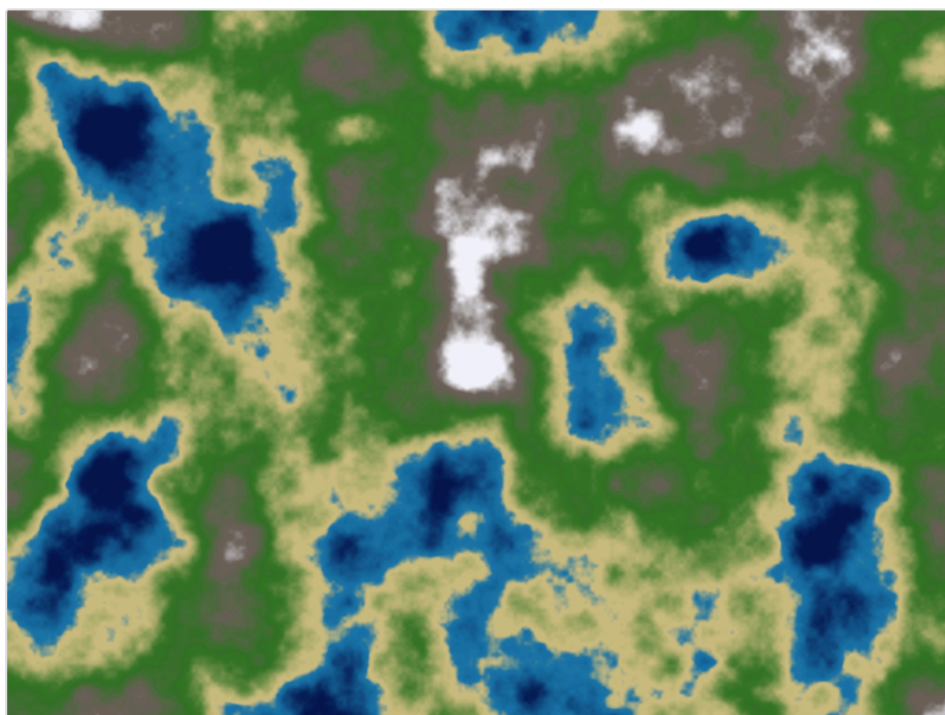
`w = fwidth(d)` is how much `d` changes from this pixel to the next. The ramp `w` to `-w` stays about **1 pixel** at both sizes.

3. What `fwidth(d)` is, and how the GPU can take a derivative of a value that each pixel computed for itself. Name the hardware fact, and the Lab 06 part where you measured it.

`fwidth(d)` is “how fast `d` changes sideways, plus how fast it changes up/down,” in neighbouring pixels.

A pixel does not know its neighbours. The GPU shades **blocks of 2×2 pixels (quads)** together, and the derivative is just the difference inside that block. Same fact as **Lab 06 Part III**, where neighbouring pixels taking different branches showed us the subgroup/quad.

Write-up 2



1. Before measuring: your predicted ratio between 8 octaves and 1, the shape of the curve, and one line of reasoning.

Predicted ratio **8x**, a **straight line**. Each extra octave is one more `value_noise` at every pixel, so eight octaves should cost eight times one.

2. Your eight measurements, the ms per octave, and whether the curve is linear. Compare with your prediction. If it is not linear, say what else the measurement is measuring.

octaves	ms median
1	0.015
2	0.023
3	0.031
4	0.040
5	0.048
6	0.057
7	0.065
8	0.075

Ratio 8 vs 1 is $0.075 / 0.015 = 5$, not 8. From 1 to 8 the extra cost is about **0.0086 ms per octave**, and the steps are almost even, so the **slope is linear**. The prediction missed a **fixed cost** (clearing, the rest of the pass, work that does not grow with octaves). The timer is the whole pass, not only the noise loop.

3. The budget in 7.6's scenario is 0.2 ms. How many octaves can you afford at 800×600 , and how many at 4K, which is $17.3\times$ the pixels? State the assumption your extrapolation makes.

At 800×600 , 8 octaves are **0.075 ms**, under 0.2 ms, so **all 8**.

Assume time grows with pixel count. Then 1 octave at 4K is about $0.015 \times 17.3 \approx 0.26$ ms, already over 0.2 ms, so **none**. That assumes every millisecond we measured scales with resolution, including the fixed cost.

4. Why your friend's GPU draws a different terrain when the hash is the chapter's $\text{fract}(\sin(\text{dot}(p, \dots)) * 43758.5453)$, and why the integer hash does not have that problem.

$\sin$ of a huge number is where GPUs disagree, so the same shader can hash to different values on two cards. Multiply, shift, and xor on integers are the same on every GPU and on the CPU, so the terrain matches.

5. Why Hermite $f^2(3 - 2f)$ and not plain f . Swap them, look at mode 2, and describe what appears.

Plain f makes a sharp crease at every grid line: mode 2 looks like a **tiled grid of squares** (diamond-shaped ridges on the cell edges). The smooth curve starts and stops with zero slope, so neighbouring cells join without that crease and the blobs look round.

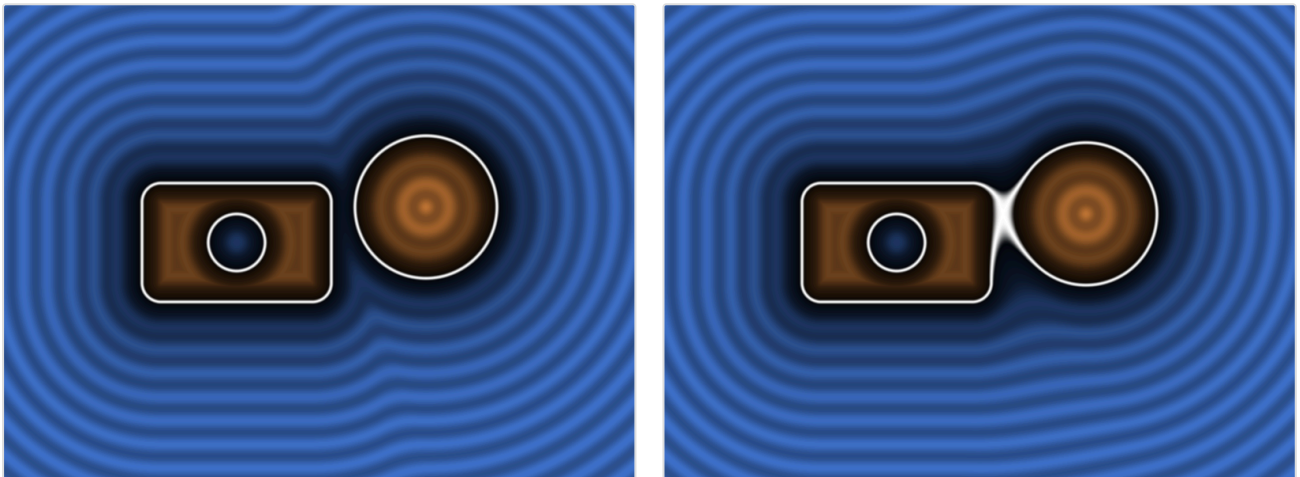
Write-up 3

1. Why $\min$ is union and $\max(d1, -d2)$ is subtraction. One sentence each, in terms of what the distance to the combined shape must be.

$\min(d1, d2)$ is union because the distance to either shape is the **smaller** of the two distances — you are as close as the nearer surface. $\max(d1, -d2)$ is subtraction because you stay outside shape 2 (need $-d2 \geq 0$) while still measuring against shape 1, so the combined distance is the **larger** of those two constraints.

2. What k does to the blend, and what it does to the contour lines in mode 1. Change it, screenshot two values, and say which one you would ship.

k is how far the melt spreads: larger k pulls the field down over a wider join, so the shapes fuse with a fatter fillet. In mode 1 the contour rings **bunch and bend** more around that join when k is large; with a small k they stay closer to hard $\min$ and look sharper at the contact.

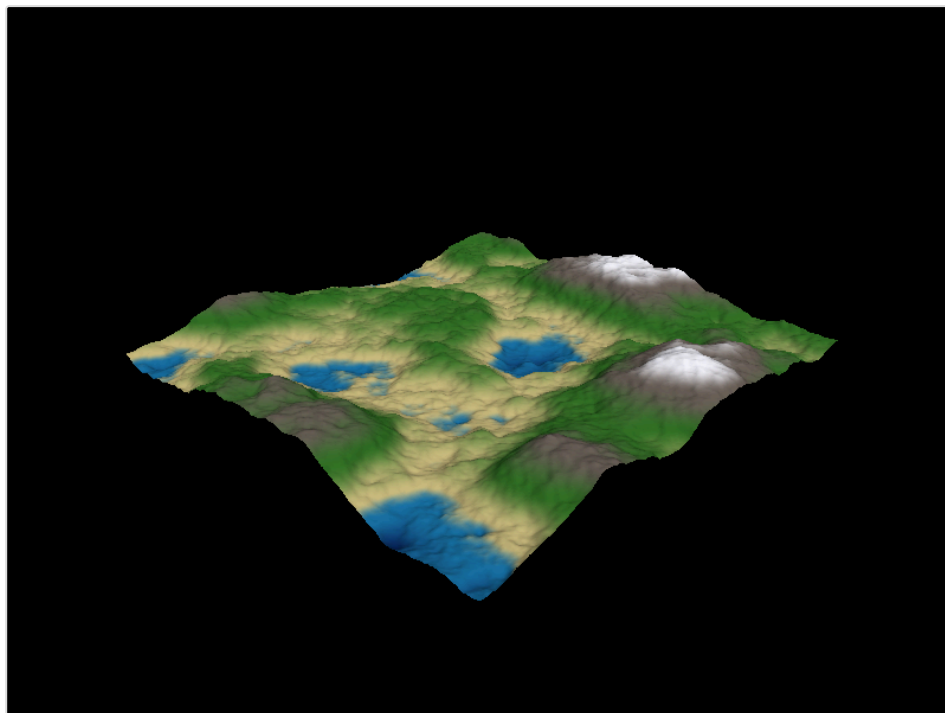


I would ship $k = 0.20$: the melt is visible and matches the reference look without turning the whole silhouette into one soft blob.

3. The field is the exact distance for a circle but only a bound after a smooth union. Quote your blend probe, say by how much it underestimates, and name one algorithm that would break because of it.

Blend probe (430, 300): field says **0.010**, nearer primitive is **0.052** away — so it underestimates by about **0.042**. A **sphere-tracing / ray-marcher** that steps by d each time can overshoot through the wall, because it trusts d as a safe step size.

Write-up 4



1. *Before measuring*: rank the five rows fastest to slowest, with one line of reasoning each in invocation counts.

Predicted order by `fbm` / `height` work (vertex rows use **3** `height()` per vertex; fragment row uses **1** `fbm` per covered fragment):

1. **vertex N=64** — $4096 \times 3 = 12288$ calls
2. **vertex N=128** — $16384 \times 3 = 49152$ calls
3. **fragment N=64** — 90772 calls ($\sim 7.4\times$ the N=64 vertex count)
4. **vertex N=256** — $65536 \times 3 = 196608$ calls
5. **vertex N=512** — $262144 \times 3 = 786432$ calls
6. Your five rows, with the invocation count beside each, against your ranking. Explain the ratio between the `N = 64` vertex row and the `N = 64` fragment row using nothing but those counts and the three calls per vertex. Then explain why the vertex rows do not quite quadruple when `N` doubles.

row	ms median	invocations
vertex N=64	0.010	12 288
fragment N=64	0.022	90 772
vertex N=128	0.027	49 152
vertex N=256	0.203	196 608
vertex N=512	2.037	786 432

Ranking by time: V64 → frag64 → V128 → V256 → V512. Prediction put frag64 after V128; on this GPU frag64 is a bit **faster** than V128 even with more calls (timer is the whole pass, not pure fbm).

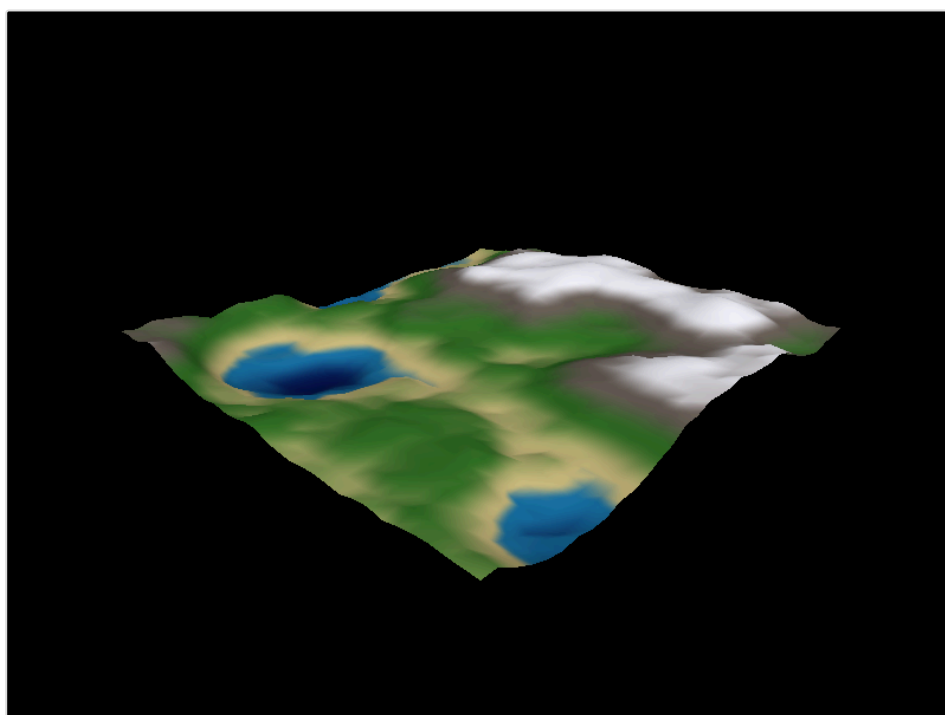
Invocation ratio frag64 / V64 = $90772 / (4096 \times 3) \approx 7.4$. If each call cost the same, fragment should be $\sim 7.4 \times$ the vertex row; measured time ratio is $0.022/0.010 = 2.2$, so other pass work and how the GPU schedules the two pipelines matter.

When N doubles, vertices (and the $3 \times$ height work) grow by $4 \times$, but the timed pass still has roughly the same screen coverage and fixed overhead, so time does not move in clean $\times 4$ steps ($0.010 \rightarrow 0.027 \rightarrow 0.203 \rightarrow 2.037$).

3. Why the finite difference normal needs three `height()` calls, and what `dFdx/dFdy` of the world position in the fragment shader would have given you instead. Say what the picture would look like.

One `height()` sets the vertex's `y`. Two more sample neighbours so you can estimate slope in `x` and `z` (the middle component of `n` is just the spacing `2e`). `dFdx / dFdy` of world position would follow the **interpolated triangle**, so the normal would be flat per face — a **faceted** terrain instead of a smooth hill.

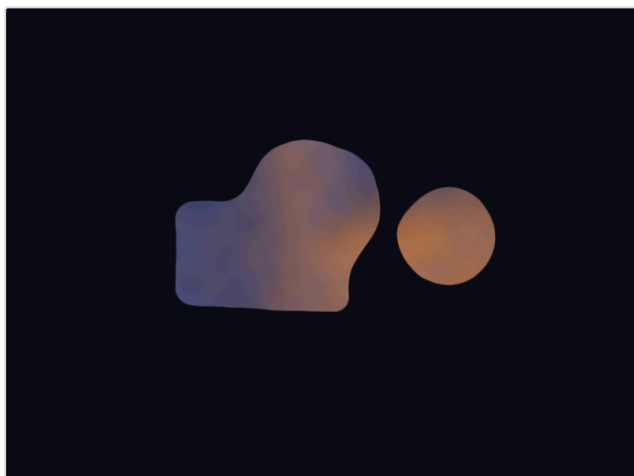
4. What happens to the silhouette when N drops below the frequency of the noise. Press [until you can see it, screenshot it, and name what the same failure is called when it happens to a texture.



The hills turn into coarse straight segments; fine bumps disappear between vertices. Same idea as texture **aliasing** (undersampling): the grid cannot represent the high frequencies in the noise.

Write-up 5

1. Your three frames, and a paragraph on how the effect works in maths: one sentence per term of the picture, naming the variable in your code.



`p` is the aspect-corrected pixel from `fragCoord`. `n = fbm(...)` is a soft noise value used to build a warped sample point `q = p + 0.08*(n-0.5)`. `d = scene(q)` smooth-unions a `sdf_rounded_box (body)` with two orbiting `sdf_circles (c1, c2)` and a mouse/`iTime` circle from `mouseP()`. `edge = smoothstep(w, -w, d)` with `w = fwidth(d)` paints an antialiased silhouette. `fill` blends blue→orange by `n`, then multiplies a second finer `fbm` for grain; `col` mixes a dark background with `fill` using `edge`.

2. The cost breakdown. Comment out your most expensive term, report the median before and after, and give it as a fraction of the frame.

Most expensive term: the second `fbm` that modulates `fill`.

	median
full shader	0.102 ms
second <code>fbm</code> commented out	0.061 ms

Saved `0.041` ms → about **40%** of the frame (`0.041 / 0.102`).

3. One thing you changed to fit the budget, and what it cost you visually.

Capped that detail `fbm` at **4** octaves (`min(u.octaves, 4u)`) instead of using the full octave count. The fill grain is a bit softer / less crispy; the melt silhouette from the SDF is unchanged. Still well under the 2.0 ms budget (**0.102** ms).

AI prompt log

Model for all entries below: **Cursor Auto (Composer)**.

1. How to do `value_noise` (guide only, no code)

- **Prompt:** Study the PDF and git state; guide Task 2 `value_noise` without giving answers or writing code.
- **Idea:** I had `pcg / hash` and a wrong stub; I needed the lattice + blend steps from the notes, not a pasted function.
- **Verification:** Followed floor/fract → `+1000` → four hashes → Hermite → bilinear mix; later checked with `make terrain` lattice **169**.

2. Check my `value_noise` body / how to blend

- **Prompt:** Validate the current body; explain how to blend.
- **Idea:** I had corners and Hermite but types/ `return 0` were wrong; I needed mix order, not a rewrite.
- **Verification:** Fixed types to `uvec2 / float`, returned nested `mix(..., w.x/w.y)`; lattice probe and image MATCH.

3. Complete `value_noise` for me

- **Prompt:** Finish the function.
- **Idea:** Blend logic was clear; I wanted a compiling version to unblock `fbm`.
- **Verification:** `make terrain`: lattice 169, one octave 84, image MATCH at 6 octaves.

4. Write-ups 1 and 2

- **Prompt:** Fill write-ups 1-2 in plain language; keep my numbers.
- **Idea:** I understood Y-flip / `fwidth` / octaves in outline; needed concise answers tied to this machine.
- **Verification:** Wrote from PDF checks + my octave table (`0.015 ... 0.075` ms); budget math uses those medians.

5. Interactive keys (0/1/2/3/S) not working

- **Prompt:** Why don't keys work under `make circle`?
- **Idea:** Panel printed `keys ...`; window drew, but input seemed dead — suspected focus/Wayland.
- **Verification:** Read `Frame.cpp` (`glfwGetKey` needs window focus); clicked the Lab window — modes/S worked.

6. Brief Part III / Task 3

- **Prompt:** Explain Part III so I can write the SDF code myself; hint at PDF equations.
- **Idea:** Knew circle SDF from Part I; needed box, rounded box, smin, and dictated `scene` order.
- **Verification:** Implemented from the brief; `make sdf` probes vs CPU, coverage 12.5%, image MATCH.

7. Validate Task 3 (first pass)

- **Prompt:** Validate 3a-d.
- **Idea:** Thought box/smin/scene were close; compile failed so I needed a checklist.
- **Verification:** AI flagged `+ r` vs `- r`, wrong smin combine, ball/hole args; fixed and re-ran panel.

8. Validate Task 3 again

- **Prompt:** Validate all of Task 3.
- **Idea:** After fixes, confirm mode 0 AA too.
- **Verification:** `make sdf --headless`: all probes OK, coverage 12.5%, MATCH.

9. Write-up 3

- **Prompt:** Complete write-up 3, same format.
- **Idea:** Understood min/max/k/bound from the panel's blend line; needed short answers + my 0.010 vs 0.052.
- **Verification:** Quoted panel bound line; mode-1 screenshots at `k=0.05` / `0.20` in report.

10. How to screenshot different `k`

- **Prompt:** How do I capture mode 1 at two `k` values?
- **Idea:** Needed a practical hot-reload + `S` workflow.
- **Verification:** Edited `k`, saved, mode 1, `S`, renamed PNGs under `report/`.

11. Brief Task 4

- **Prompt:** Brief TASK 4.
- **Idea:** Knew `fbm` from Part II; needed where height/normal/bands go (vert vs frag).
- **Verification:** Implemented `height` / central differences / `bands` + Lambert; later MATCH at N=256.

12-13. Validate Task 4 (twice)

- **Prompt:** Validate heightmap code; then again after fix.
- **Idea:** First version forgot `xz*1.5 + offset` and `* u.knobf`; second should match reference.
- **Verification:** First DIFFERS / wrong normals; after restoring the PDF `height()` line: normals `(149,244,81)`, coverage 18.1%, MATCH.

14. Write-up 4

- **Prompt:** Do write-up 4, same format.
- **Idea:** Had the five timing rows; needed invocation ranking and N=32 aliasing wording.
- **Verification:** Used my panel timings; attached `heightmap_N32.png` for the silhouette point.

15. Is `heightmap_N32.png` usable?

- **Prompt:** Check that screenshot for write-up 4.4.
- **Idea:** Wanted to know if mode 0 at N=32 shows the failure clearly enough.
- **Verification:** Usable (coarse mesh); mode 2 would be clearer — kept the N=32 shot in the report.

16-17. Task 5 idea (ShaderToy link + 3 recommendations)

- **Prompt:** Is XsK3RR OK? Then recommend ~3 original ideas.
- **Idea:** Needed something that greps `sdf_ / fbm / iMouse / iMode` and stays <2 ms — not multipass.
- **Verification:** Rejected XsK3RR (Buffer A / not original); picked melting-blobs recipe from the three options.

18. Implement melting blobs (Task 5)

- **Prompt:** Do option 1 for me.
- **Idea:** Wanted a complete `mytoy.frag` meeting all five rules.
- **Verification:** `make toy`: budget PASS (~0.102 ms), all greps yes, TODO removed, `toy_0/1/2.png` written.

19. Write-up 5

- **Prompt:** Complete write-up 5.
- **Idea:** Needed cost split for the second `fbm` and a budget trade-off sentence.
- **Verification:** Measured full vs second- `fbm` commented (0.102 → 0.061 ms ≈ 40%); restored shader; frames linked in report.

20. AI prompt log

- **Prompt:** Complete the AI prompt log in `report.md`.
- **Idea:** Lab §9 wants prompt / model / idea / verification per prompt — empty “wrote noise for me” entries fail.
- **Verification:** Entries above match this session; each ties to a panel check, MATCH, or a measured number in the write-ups.